

Evaluación Parcial N°3

Diseño de Arquitectura de Microservicios

SmartLogix — Plataforma Inteligente para la Gestión Logística de eCommerce

Asignatura: Desarrollo Fullstack III — DSY1106

Integrantes: Catrina Corral

Anais Aravena

Fernanda Manríquez

Sección: 002D

Fecha: Junio 2026

2. Índice

1. Portada

2. Índice

3. Contexto del Caso

4. Arquitectura Microservicio

5. Patrones de Arquitectura Aplicados

6. Comunicación entre Microservicios

7. Evaluación de Patrones Implementados

8. Testing: Pruebas Unitarias y Cobertura

9. Evidencia de versionamiento y branching en GitHub

10. Evidencia de creación y publicación con Docker

Bibliografía

Conclusión

3. Contexto del Caso

SmartLogix surge como una solución tecnológica orientada a optimizar la gestión logística de pequeñas y medianas empresas dentro del ecosistema eCommerce. Actualmente, muchas organizaciones operan bajo arquitecturas monolíticas, lo que genera problemas como falta de escalabilidad, dificultades en la integración con otros sistemas y errores en la sincronización de información.

Estas limitaciones afectan directamente la eficiencia operativa, provocando retrasos en la gestión de pedidos, inconsistencias en el inventario y una mala experiencia para el cliente final. Frente a este escenario, se propone una arquitectura basada en microservicios, donde el sistema se divide en componentes independientes que permiten una mayor flexibilidad, escalabilidad y mantenibilidad.

Los principales componentes del sistema son:

- Microservicio de Inventario, encargado de controlar el stock en tiempo real.
- Microservicio de Pedidos, responsable de validar y gestionar órdenes.
- Microservicio de Envíos, encargado de la logística de despacho.
- Microservicio de Usuarios, encargado de la validación de credenciales, generación de tokens JWT y control de acceso al sistema.
- Servicio de Notificaciones, encargado de enviar avisos al usuario sobre el estado de sus pedidos, confirmaciones y actualizaciones relevantes del sistema.

Esta arquitectura permite responder de manera eficiente a los requerimientos del cliente, facilitando la integración con plataformas externas y mejorando el rendimiento del sistema.

4. Arquitectura Microservicio

La arquitectura SmartLogix se implementa mediante componentes frontend y backend desacoplados. El frontend React consume los endpoints expuestos por el API Gateway, que actúa como punto único de entrada hacia los microservicios. Eureka Server permite registrar y descubrir servicios, mientras que cada microservicio mantiene su propia responsabilidad de negocio y su propia persistencia.

El backend evidencia un API Gateway en el puerto 8080, Eureka Server en el puerto 8761, y microservicios independientes para Usuarios, Inventario, Pedidos, Envíos y Notificaciones. Además, se integra RabbitMQ para eventos de stock crítico y Docker Compose para levantar la infraestructura local.

Componente	Responsabilidad	Persistencia / integración
Frontend React	Interfaz de usuario para login, inventario, pedidos, envíos y notificaciones	Consume API REST vía API Gateway
API Gateway	Punto único de entrada, rutas /api/** y autenticación JWT	Eureka + JWT
Usuarios	Gestión de usuarios, credenciales y roles	PostgreSQL baseusuarios
Inventario	Productos, bodegas, proveedores y stock crítico	PostgreSQL baseinventario + RabbitMQ producer

Pedidos	Creación de pedidos, validaciones y boletas	PostgreSQL basepedidos + OpenFeign
Envíos	Despacho, trazabilidad y estados de envío	PostgreSQL baseenvios + Feign + Circuit Breaker
Notificaciones	Avisos de pedidos y alertas de stock crítico	PostgreSQL basenotificaciones + RabbitMQ consumer
Transportista externo	Servicio simulado para disponibilidad de despacho	httpbin.org mediante Feign

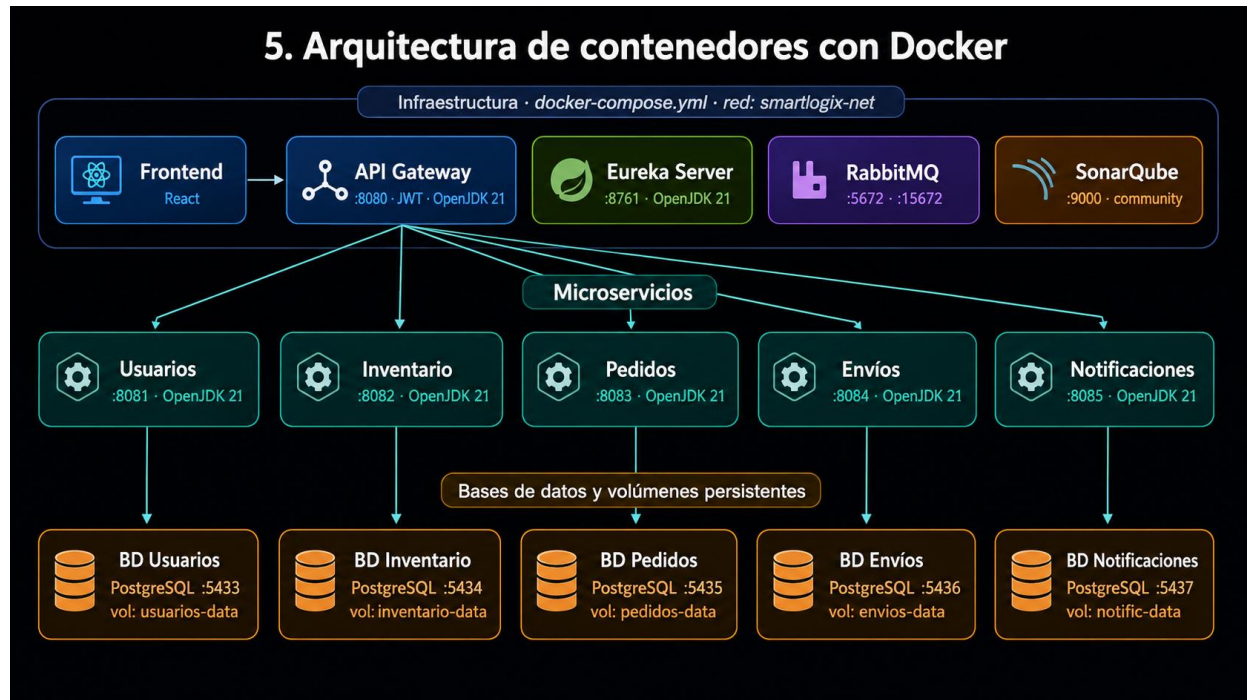


Figura 4.1. Arquitectura de microservicios, contenedores y persistencia por servicio.

4.1 Persistencia de datos

La persistencia se garantiza mediante PostgreSQL y JPA. Cada microservicio posee su propia base de datos, aplicando el patrón Database per Service. Esta separación evita que un servicio acceda directamente a la base de otro, mejora el aislamiento de responsabilidades y facilita el mantenimiento independiente de cada módulo.

La arquitectura utiliza Docker para contenerizar cada microservicio junto con sus dependencias y su base de datos PostgreSQL, asegurando consistencia entre entornos de desarrollo, pruebas y producción. Cada servicio se empaqueta mediante imágenes, contenedores y Dockerfile, permitiendo ejecutar aplicaciones de forma estandarizada y portable entre distintos entornos (Richardson, 2018). Como se observa en el diagrama, los microservicios de Usuarios, Inventario, Pedidos, Envíos y Notificaciones se ejecutan de forma desacoplada en contenedores independientes, facilitando el mantenimiento y la evolución del sistema.

5. Patrones de Arquitectura Aplicados

En la solución implementada se utilizan patrones alineados con los requerimientos del sistema SmartLogix. El API Gateway centraliza el acceso y la autenticación mediante JWT; Eureka Server permite el registro y descubrimiento de servicios; Repository Pattern desacopla la lógica de negocio de la persistencia; Factory Method permite crear pedidos y envíos normales o express; Circuit Breaker se aplica en Envíos para proteger la comunicación con el transportista externo; RabbitMQ permite comunicación asíncrona para eventos de stock crítico; y Database per Service asegura bases PostgreSQL independientes por microservicio.

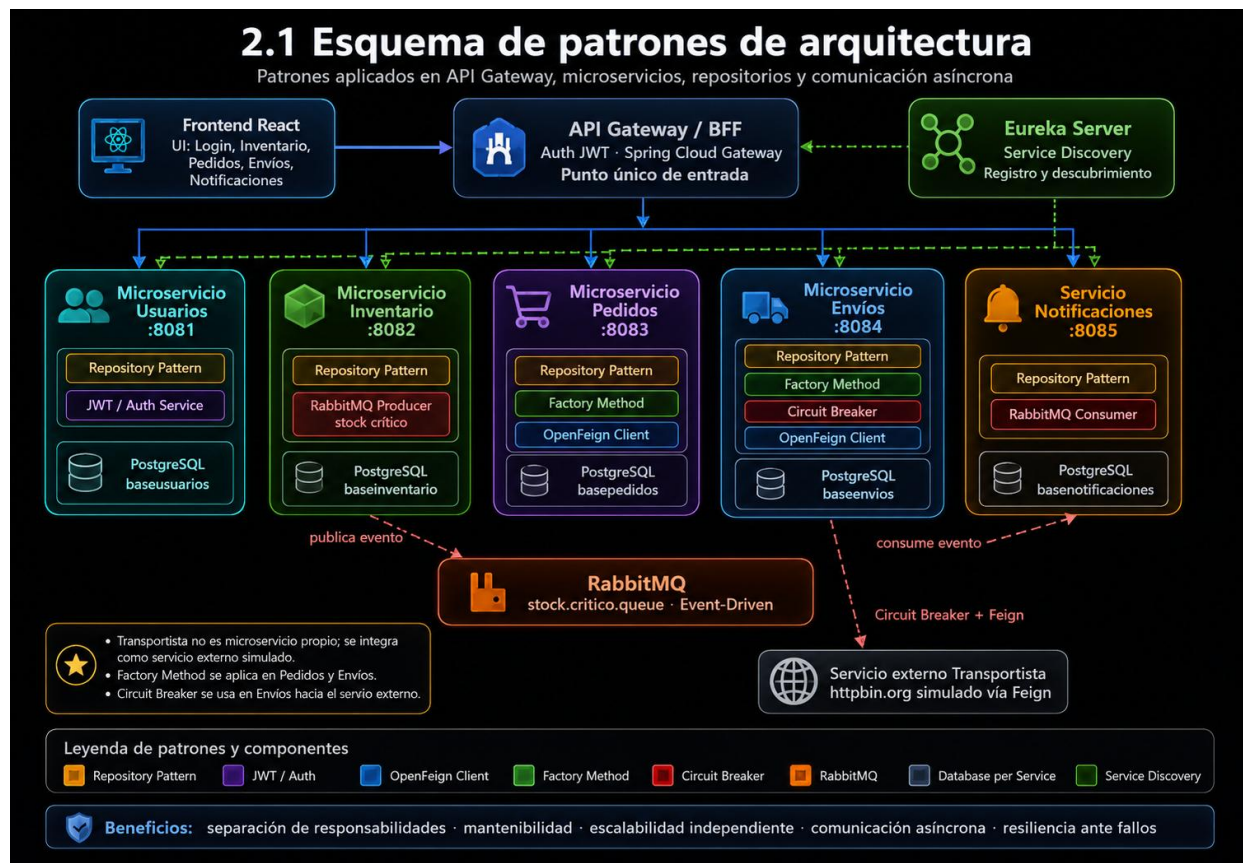


Figura 5.1. Patrones aplicados en los microservicios SmartLogix.

5.2 Justificación de los patrones seleccionados

Se selecciona el patrón API Gateway debido a que permite centralizar el acceso al sistema SmartLogix, evitando que los clientes deban conocer la ubicación de cada microservicio, además de facilitar la autenticación mediante JWT y el control de solicitudes (Mostapha, 2025). En SmartLogix, el API Gateway recibe las solicitudes del frontend y las dirige al microservicio correspondiente, como Pedidos o Inventario.

El Repository Pattern permite desacoplar la lógica de negocio de la base de datos en los microservicios de Inventario, Pedidos, Envíos, Usuarios y Notificaciones, facilitando la mantenibilidad del sistema y

permitiendo posibles cambios tecnológicos sin afectar la arquitectura (Kranio, 2024). Por ejemplo, el microservicio de Inventario utiliza este patrón para acceder a la base de datos sin exponer directamente la lógica de persistencia.

El Factory Method se utiliza en el procesamiento de pedidos y la coordinación de envíos, permitiendo gestionar distintos tipos de pedidos y envíos sin el uso excesivo de estructuras condicionales, mejorando la extensibilidad del sistema (Gamma et al., 1994). En el sistema, este patrón permite crear distintos tipos de pedidos y envíos, por ejemplo flujos normales o express, sin modificar la lógica principal.

El Circuit Breaker se aplica en el microservicio de Envíos, específicamente en la comunicación con servicios externos como transportistas, permitiendo detectar fallos y retornar respuestas alternativas, evitando fallos en cascada (AWS, s.f.). Por ejemplo, si el servicio externo de transportistas falla, el sistema responde con un estado alternativo sin interrumpir el flujo del pedido.

Finalmente, el patrón Database per Service permite que cada microservicio gestione su propio PostgreSQL, asegurando independencia, aislamiento y escalabilidad del sistema (AWS, s.f.). Por ejemplo, cada microservicio accede únicamente a su propia base de datos, sin depender de las bases de datos de otros servicios.

El patrón Service Discovery se aplica mediante Eureka Server. Esto permite que el API Gateway y los microservicios se registren y descubran utilizando nombres lógicos de servicio, reduciendo la dependencia de direcciones fijas y facilitando la escalabilidad del entorno.

El patrón Event-Driven se aplica mediante RabbitMQ. Inventario publica un evento cuando detecta stock crítico y Notificaciones consume dicho evento para generar la alerta correspondiente, evitando acoplamiento directo entre ambos servicios.

6. Comunicación entre Microservicios

La comunicación entre los componentes se implementa mediante una combinación de comunicación síncrona REST/HTTP, clientes declarativos OpenFeign y mensajería asíncrona con RabbitMQ. Esta estructura permite responder de forma inmediata en operaciones críticas y desacoplar procesos que pueden ejecutarse por eventos.

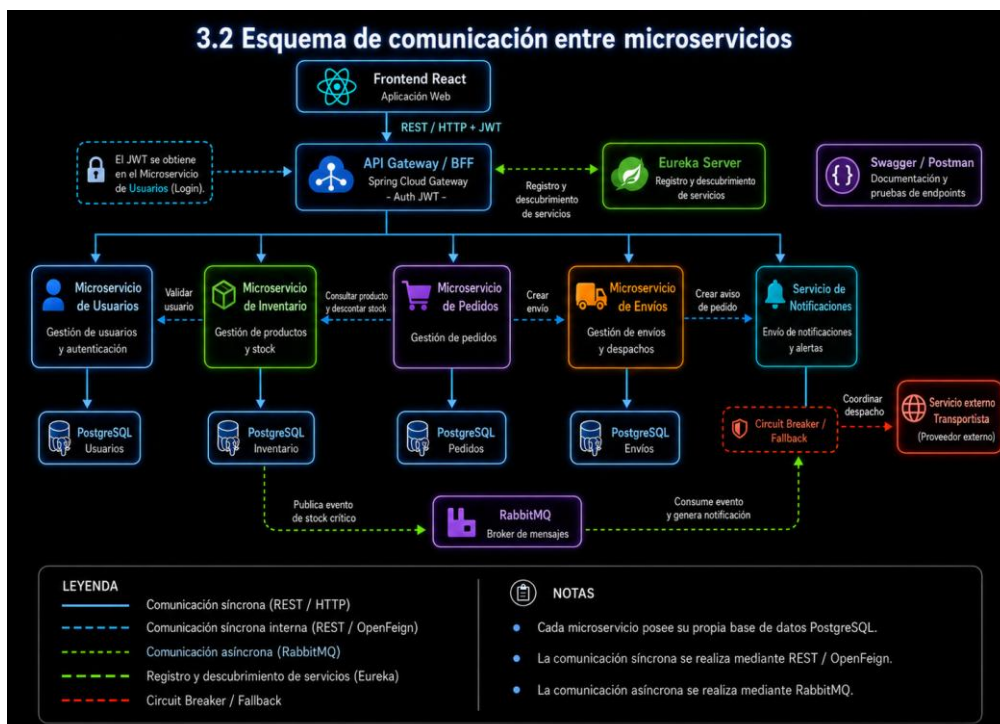


Figura 6.1. Comunicación entre microservicios según el backend implementado.

La arquitectura utiliza comunicación síncrona basada en REST/HTTP para operaciones que requieren respuesta inmediata, como validación de usuarios y verificación de pedidos. Para ello, se utiliza Spring Cloud OpenFeign, facilitando la comunicación entre microservicios mediante interfaces declarativas.

El microservicio de pedidos coordina el flujo principal del sistema, comunicándose con Inventario, Usuarios y Envíos para validar información y gestionar órdenes. Por su parte, el servicio de Notificaciones informa al usuario sobre eventos relevantes, como el estado de los pedidos y las alertas de stock crítico.

Adicionalmente, se implementa comunicación asíncrona mediante RabbitMQ, donde el Microservicio de Inventario publica eventos de stock crítico en la cola `stock.critico.queue`, los cuales son consumidos por el Servicio de Notificaciones para generar las alertas correspondientes.

El servicio externo de transportista no corresponde a un microservicio propio del sistema; se integra desde el microservicio de Envíos mediante Feign y se protege con Circuit Breaker para evitar fallos en cascada.

7. Evaluación de Patrones Implementados

Los patrones seleccionados fueron validados durante la implementación del sistema SmartLogix, demostrando que cada uno resuelve directamente una problemática concreta del caso.

API Gateway resolvió el problema de acceso distribuido: el frontend React se comunica con un único punto de entrada en el puerto 8080, que valida el token JWT mediante `JwtAuthFilter` antes de enrutar la solicitud al microservicio correspondiente. Esto eliminó la necesidad de exponer cada servicio directamente al frontend.

Repository Pattern permitió desacoplar la lógica de negocio de la persistencia en los microservicios. Gracias a las interfaces JPA Repository, fue posible realizar pruebas unitarias con Mockito sin necesidad de levantar la base de datos, lo que facilita alcanzar la cobertura mínima exigida.

Factory Method se implementó en los flujos de creación de pedidos y envíos, permitiendo instanciar alternativas normales o express según los parámetros recibidos sin modificar la lógica principal del servicio. Esto resuelve el problema de extensibilidad ante nuevos tipos de operación logística.

Circuit Breaker se aplicó en el microservicio de Envíos para la comunicación con el servicio externo de transportista mediante Feign. Si el servicio externo no responde, el sistema activa el fallback y retorna un estado alternativo, evitando que un fallo externo interrumpa el flujo completo del pedido.

Database per Service garantizó la independencia entre microservicios, con cada uno gestionando su propio PostgreSQL en un contenedor separado. Esto resuelve el problema de acoplamiento de datos presente en una arquitectura monolítica.

RabbitMQ / Event-Driven complementó la arquitectura con comunicación asíncrona: el microservicio de Inventario publica eventos en la cola stock.critico.queue cuando el stock baja del umbral definido, y el servicio de Notificaciones los consume para generar alertas automáticamente sin bloquear el flujo principal.

Eureka Server resolvió el problema de acoplamiento entre servicios al registrar dinámicamente cada microservicio, permitiendo que el API Gateway enrute mediante nombres lógicos como lb://serviciousuarios en lugar de depender de IPs fijas.

7.1 Evaluación complementaria: seguridad, privacidad y sostenibilidad

La seguridad se implementa mediante autenticación basada en tokens JWT. Al iniciar sesión, cada usuario recibe un token con sus roles y permisos específicos, el cual es validado por cada microservicio antes de procesar la solicitud.

Se aplica el principio de mínimo privilegio, donde cada usuario solo puede acceder a las funciones correspondientes a su rol (por ejemplo, un operador de bodega solo puede actualizar stock).

Toda la comunicación se protege mediante protocolos seguros HTTPS/TLS, y cada transacción se registra con información como usuario, timestamp e IP de origen, permitiendo la detección de posibles anomalías.

Los datos de clientes (direcciones, pedidos, contactos) se gestionan de forma segmentada: cada microservicio accede solo a la información que necesita. El área de envíos no puede ver datos financieros. Se cumple con la normativa de privacidad vigente en Chile.

La arquitectura de microservicios permite que cada componente evolucione de forma independiente, reduciendo la deuda técnica y facilitando la incorporación de nuevas tecnologías sin afectar el sistema completo. Además, el uso de contenedores permite optimizar el consumo de recursos, ejecutando únicamente los servicios necesarios según la demanda. Esto permite reducir el uso energético en períodos de baja carga y mejorar la eficiencia operativa del sistema a largo plazo.

8. Testing: Pruebas Unitarias y Cobertura

8.1 Estrategia de pruebas unitarias

Para garantizar la calidad del sistema SmartLogix, se implementaron pruebas unitarias en la capa service de los microservicios principales utilizando JUnit 5 y Mockito. Las pruebas aíslan la lógica de negocio

mediante mocks de repositorios, clientes Feign y dependencias externas, evitando depender de la base de datos o de servicios externos durante la ejecución de los tests.

8.2 Microservicio Usuarios

Clase service considerada: UsuarioServiceImpl.

Métodos testeados: guardar, buscarPorUsername y existePorUsername.

Se valida la gestión de usuarios, la búsqueda por nombre de usuario y la verificación de existencia de credenciales registradas.

Pruebas identificadas en el backend:

- guardar_debeRetornarUsuarioGuardado — verifica que al guardar un usuario se retorna el objeto guardado correctamente.
- buscarPorUsername_debeRetornarUsuarioCuandoExiste — valida que se encuentra el usuario cuando el username existe.
- buscarPorUsername_debeRetornarVacioCuandoNoExiste — valida que retorna vacío cuando el username no existe.
- existePorUsername_debeRetornarTrueCuandoExiste — confirma que retorna true cuando el usuario ya está registrado.
- existePorUsername_debeRetornarFalseCuandoNoExiste — confirma que retorna false cuando el usuario no existe.

```
18 @ExtendWith(MockitoExtension.class)
19 class ServiciosUsuariosApplicationTests {
20
21     @Mock
22     private UsuarioRepository usuarioRepository;
23
24     @InjectMocks
25     private UsuarioServiceImpl usuarioService;
26
27     private Usuario usuario;
28
29     @BeforeEach
30     void setUp() {
31         usuario = new Usuario();
32         usuario.setId(id: 1L);
33         usuario.setUsername(username: "anaís");
34         usuario.setPassword(password: "password123");
35         usuario.setRol(rol: "USER");
36         usuario.setCorreo(correo: "anaís@gmail.com");
37     }
38
39     @Test
40     void guardar_debeRetornarUsuarioGuardado() {
41         when(usuarioRepository.save(usuario)).thenReturn(usuario);
42
43         Usuario resultado = usuarioService.guardar(usuario);
44
45         assertNotNull(resultado);
46         assertEquals("anaís", resultado.getUsername());
47         verify(usuarioRepository, times(1)).save(usuario);
48     }
49 }
```

```
C:\Windows\System32\cmd.exe - .\mvnw.cmd test
[INFO] --- resources:3.3.1:testResources (default-testResources) @ serviciuousuarios ---
[INFO] skip non existing resourceDirectory C:\Users\anaia\Downloads\smartlogix_backend_aravena_corral_manriquez-mainaja\smartlogix_backend_aravena_corral_manriquez-mainaja\proyectoSmartLogix\serviciuousuarios\src\test\resources
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ serviciuousuarios ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.5.5:test (default-test) @ serviciuousuarios ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.smartlogix.serviciuousuarios.ServiciuousuariosApplicationTests
Mockito is currently self-attaching to enable the inline mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\anaia\m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.368 s -- in com.smartlogix.serviciuousuarios.ServiciuousuariosApplicationTests
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 8.230 s
[INFO] Finished at: 2026-06-16T19:41:41-04:00
[INFO] -----
```

8.3 Microservicio Inventario

Clase service considerada: ProductoServiceImpl.

Métodos testeados: guardar, listarTodos, buscarPorId, eliminar, existePorNombre, descontarStock y listarStockCriticoAlerta.

Se valida la creación de productos, normalización del tipo de stock, consulta de inventario, descuento de stock y detección de productos bajo el umbral crítico.

Pruebas identificadas en el backend:

- guardar_debeRetornarProductoGuardado — verifica que al guardar un producto se retorna el objeto guardado con nombre y tipo de stock correctos.
- guardar_debeNormalizarTipoStockAMayusculas — valida que el tipo de stock ingresado en minúsculas es normalizado automáticamente a mayúsculas.
- guardar_debeLanzarExcepcionCuandoTipoStockEsInvalido — confirma que se lanza una excepción si el tipo de stock no es válido y no se guarda en la base de datos.
- listarTodos_debeRetornarListaDeProductos — verifica que se retorna correctamente la lista de todos los productos registrados.
- buscarPorId_debeRetornarProductoCuandoExiste — valida que se encuentra el producto cuando el ID existe en el repositorio.
- buscarPorId_debeRetornarVacioCuandoNoExiste — confirma que retorna vacío cuando el ID no corresponde a ningún producto.
- eliminar_debeEliminarProducto — verifica que el método deleteById es invocado correctamente al eliminar un producto.
- existePorNombre_debeRetornarTrueCuandoExiste — valida que retorna true cuando ya existe un producto con ese nombre registrado.

- `descontarStock_debeDescontarCantidadCorrectamente` — verifica que la cantidad del producto se reduce correctamente al descontar stock.
- `descontarStock_debeLanzarExcepcionCuandoStockInsuficiente` — confirma que se lanza una excepción cuando la cantidad a descontar supera el stock disponible.
- `descontarStock_debeLanzarExcepcionCuandoCantidadEsCero` — valida que no se permite descontar una cantidad de cero unidades.
- `listarStockCriticoAlerta_debeRetornarProductosConStockBajo` — verifica que se retornan los productos cuya cantidad es inferior al stock mínimo definido.

```

@ExtendWith(MockitoExtension.class)
class ServicioInventarioApplicationTests {

    @Mock
    private ProductoRepository productoRepository;

    @Mock
    private RabbitTemplate rabbitTemplate;

    @InjectMocks
    private ProductoServiceImpl productoService;

    private Producto producto;

    @BeforeEach
    void setUp() {
        producto = new Producto();

        producto.setId(id: 1L);
        producto.setNombre(nombre: "Laptop");
        producto.setDescripcion(descripcion: "Laptop gamer");
        producto.setCantidad(cantidad: 10);
        producto.setPrecio(precio: 999.99);
        producto.setTipoStock(tipoStock: "VENTA");
        producto.setStockMinimo(stockMinimo: 2);
    }

    @Test
    void guardar_debeRetornarProductoGuardado() {
        when(productoRepository.save(any(Producto.class))).thenReturn(producto);

        Producto resultado = productoService.guardar(producto);
    }
}

```

```

C:\Windows\System32\cmd.exe - .\mvnw.cmd test
[INFO] --- resources:3.3.1:testResources (default-testResources) @ servicioinventario ---
[INFO] skip non existing resourceDirectory C:\Users\anaia\Downloads\smartlogix_backend_aravena_corral_manriquez-main\smartlogix_backend_aravena_corral_manriquez-main\projectoSmartLogix\servicioinventario\src\test\resources
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ servicioinventario ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.5.5:test (default-test) @ servicioinventario ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running com.smartlogix.servicioinventario.ServicioInventarioApplicationTests
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\anaia\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.156 s -- in com.smartlogix.servicioinventario.ServicioInventarioApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 8.671 s
[INFO] Finished at: 2026-06-16T19:40:14-04:00
[INFO]

```

8.4 Microservicio Pedidos

Clase service considerada: PedidoServiceImpl.

Métodos testeados: crear, listar y buscarPorId.

Se valida la creación de pedidos con mocks de usuario, producto, stock y dirección, además de escenarios de error por datos inválidos o inexistentes.

Pruebas identificadas en el backend:

- crear_debeCrearPedidoCorrectamente — verifica que al crear un pedido con datos válidos se retorna el pedido guardado con estado CREADO.
- crear_debeLanzarExcepcionCuandoUsuarioNoExiste — confirma que se lanza una excepción cuando el usuario no es encontrado mediante el cliente Feign.
- crear_debeLanzarExcepcionCuandoProductoNoExiste — valida que se lanza una excepción cuando el producto consultado al microservicio de inventario no existe.
- crear_debeLanzarExcepcionCuandoStockInsuficiente — verifica que se lanza una excepción cuando la cantidad solicitada supera el stock disponible del producto.
- crear_debeLanzarExcepcionCuandoCantidadEsCero — confirma que no se permite crear un pedido con cantidad igual a cero.
- crear_debeLanzarExcepcionCuandoDireccionEsVacía — valida que se lanza una excepción cuando la dirección de envío está vacía.
- listar_debeRetornarListaDePedidos — verifica que se retorna correctamente la lista de todos los pedidos registrados.
- buscarPorId_debeRetornarPedidoCuandoExiste — valida que se encuentra el pedido cuando el ID existe en el repositorio.
- buscarPorId_debeRetornarVacioCuandoNoExiste — confirma que retorna vacío cuando el ID no corresponde a ningún pedido registrado.

```

class ServicioPedidosApplicationTests {

    void setUp() {
        usuario = new Usuario();
        usuario.setId(id: 1L);
        usuario.setUsername(username: "anaís");

        producto = new Producto();
        producto.setId(id: 1L);
        producto.setNombre(nombre: "Laptop");
        producto.setCantidad(cantidad: 10);
        producto.setPrecio(precio: 999.99);

        pedido = new Pedido();
        pedido.setUsername(username: "anaís");
        pedido.setProductoId(productoId: 1L);
        pedido.setCantidad(cantidad: 2);
        pedido.setDireccionEnvio(direccionEnvio: "Calle 123");
        pedido.setTipo(tipo: "NORMAL");
        pedido.setEstado(estado: "CREADO");
    }

    @Test
    void crear_debeCrearPedidoCorrectamente() {
        Pedido pedidoGuardado = new Pedido();
        pedidoGuardado.setId(id: 1L);
        pedidoGuardado.setUsername(username: "anaís");
        pedidoGuardado.setProductoId(productoId: 1L);
        pedidoGuardado.setCantidad(cantidad: 2);
        pedidoGuardado.setDireccionEnvio(direccionEnvio: "Calle 123");
        pedidoGuardado.setEstado(estado: "CREADO");
        pedidoGuardado.setTipo(tipo: "NORMAL");
    }
}

```

```

C:\Windows\System32\cmd.exe - .\mvnw.cmd test
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.5.5:test (default-test) @ serviciopedidos ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.smartlogix.serviciopedidos.ServicioPedidosApplicationTests
Mockito is currently self-attaching to enable the inline mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org.mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\anaís\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.134 s -- in com.smartlogix.serviciopedidos.ServicioPedidosApplicationTests
[INFO] Results:
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 9.958 s
[INFO] Finished at: 2026-06-16T19:41:33-04:00
[INFO]

```

8.5 Microservicio Envíos

Clase service considerada: EnvioServiceImpl.

Métodos testeados: buscarPorId, buscarPorPedidoId, listar, actualizarEstado y fallbackCrear.

Se valida la trazabilidad de envíos, la actualización de estados permitidos y el comportamiento del fallback cuando el servicio externo no responde.

Pruebas identificadas en el backend:

- `buscarPorId_debeRetornarEnvioCuandoExiste` — verifica que se retorna el envío correctamente cuando el ID existe en el repositorio.
- `buscarPorId_debeRetornarVacioCuandoNoExiste` — confirma que retorna vacío cuando el ID no corresponde a ningún envío registrado.
- `buscarPorPedidoId_debeRetornarEnvioCuandoExiste` — valida que se encuentra el envío asociado a un pedido cuando el `pedidoId` existe.
- `listar_debeRetornarListaDeEnvios` — verifica que se retorna correctamente la lista de todos los envíos registrados.
- `actualizarEstado_debeActualizarEstadoCorrectamente` — confirma que el estado del envío se actualiza a `EN_CAMINO` y se guarda correctamente.
- `actualizarEstado_debeLanzarExcepcionCuandoEstadoEsInvalido` — valida que se lanza una excepción cuando el estado ingresado no es un valor permitido.
- `actualizarEstado_debeLanzarExcepcionCuandoEstadoEsNulo` — confirma que se lanza una excepción cuando el estado enviado es nulo.
- `actualizarEstado_debeLanzarExcepcionCuandoEnvioNoExiste` — verifica que se lanza una excepción cuando el ID del envío a actualizar no existe.
- `fallbackCrear_debeCrearEnvioConEstadoNoDisponible` — valida que cuando el servicio externo de transportista falla, el fallback crea el envío con estado `SERVICIO_NO_DISPONIBLE`.

The screenshot shows an IDE window titled 'proyectoSmartLogix'. The Explorer panel on the left shows a project structure with folders like '.github', 'apigateway', 'eureka-server', 'servicioenvios', and 'src'. The main editor displays the file 'ServicioenviosApplicationTests.java'. The code includes imports for Mockito and the service interface, followed by mock setup for 'EnvioRepository', 'EnvioFactory', and 'TransportistaClient'. A test method 'buscarPorId_debeRetornarEnvioCuandoExiste()' is shown, which creates an 'Envio' object with specific attributes and uses the service's 'buscarPorId' method to verify the result.

```

@ExtendWith(MockitoExtension.class)
class ServicioenviosApplicationTests {

    @Mock
    private EnvioRepository envioRepository;

    @Mock
    private EnvioFactory envioFactory;

    @Mock
    private TransportistaClient transportistaClient;

    @InjectMocks
    private EnvioServiceImpl envioService;

    private Envio envio;

    @BeforeEach
    void setUp() {
        envio = new Envio();
        envio.setId(1L);
        envio.setPedidoId(1L);
        envio.setDireccion(direccion: "Calle 123");
        envio.setEstado(estado: "PENDIENTE");
    }

    @Test
    void buscarPorId_debeRetornarEnvioCuandoExiste() {
        when(envioRepository.findById(id: 1L)).thenReturn(Optional.of(envio));

        Optional<Envio> resultado = envioService.buscarPorId(id: 1L);
    }
}

```



```
C:\Windows\System32\cmd.exe - .\mvnw.cmd test
[INFO] --- resources:3.3.1:testResources (default-testResources) @ servicioenvios ---
[INFO] skip non existing resourceDirectory C:\Users\anaia\Downloads\smartlogix_backend_aravena_corral_manriquez-mainaja\smartlogix_backend_aravena_corral_manriquez-mainaja\projectoSmartlogix\servicioenvios\src\test\resources
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ servicioenvios ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.5.5:test (default-test) @ servicioenvios ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running com.smartlogix.servicioenvios.ServicioenviosApplicationTests
Mockito is currently self-attaching to enable the inline mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\anaia\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.288 s -- in com.smartlogix.servicioenvios.ServicioenviosApplicationTests
[INFO] Results:
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 7.502 s
[INFO] Finished at: 2026-06-16T19:39:49-04:00
[INFO]
```

8.6 Microservicio Notificaciones

Clase service considerada: NotificacionServiceImpl.

Métodos testeados: crear, listar, listarPorUsuario, buscarPorId y eliminar.

Se valida la creación y consulta de notificaciones, la eliminación controlada y la gestión de tipos de notificación válidos.

Pruebas identificadas en el backend:

- `crear_debeRetornarNotificacionCreada` — verifica que al crear una notificación con tipo PEDIDO se retorna el objeto guardado correctamente.
- `crear_debeLanzarExcepcionCuandoTipoEsInvalido` — confirma que se lanza una excepción cuando el tipo de notificación no es válido y no se guarda.
- `crear_debeAceptarTipoENVIO` — valida que el servicio acepta y guarda correctamente notificaciones de tipo ENVIO.
- `crear_debeAceptarTipoSTOCK` — valida que el servicio acepta y guarda correctamente notificaciones de tipo STOCK.
- `listar_debeRetornarListaDeNotificaciones` — verifica que se retorna correctamente la lista de todas las notificaciones registradas.
- `listarPorUsuario_debeRetornarNotificacionesDelUsuario` — valida que se retornan únicamente las notificaciones asociadas al `usuarioId` indicado.
- `buscarPorId_debeRetornarNotificacionCuandoExiste` — confirma que se encuentra la notificación correctamente cuando el ID existe.
- `buscarPorId_debeRetornarVacioCuandoNoExiste` — valida que retorna vacío cuando el ID no corresponde a ninguna notificación.
- `eliminar_debeEliminarNotificacionCuandoExiste` — verifica que la notificación es eliminada correctamente cuando el ID existe en el repositorio.

The screenshot shows an IDE interface with a dark theme. The top bar includes a menu (File, Edit, Selection, View, Go, Run, ...), a search bar, and a 'Sign In' button. The Explorer panel on the left shows a project structure with folders like .github, apigateway, eurekaserver, servicioenvios, servicioinventario, and servicio notificaci..., and files like .mvn, src, main, test, and pom.xml. The main editor displays a Java file named 'ServicionotificacionesApplicationTests.java'. The code includes imports for Mockito and a test class 'NotificacionServiceImplTest' with methods 'setUp()' and 'crear_debeRetornarNotificacionCreada()'. The bottom status bar shows 'Ln 76, Col 6' and 'Spaces: 4'. The taskbar at the bottom displays various application icons and the system clock showing 10°C.

```
C:\Windows\System32\cmd.exe - .\mvnw.cmd test
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO] --- compiler:3.14.1:compile (default-compile) @ servicionotificaciones ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.3.1:testResources (default-testResources) @ servicionotificaciones ---
[INFO] skip non existing resourceDirectory C:\Users\anaia\Downloads\smartlogix_backend_aravena_corral_manriquez-mainaja\smartlogix_backend_aravena_corral_manriquez-main
project\SmartLogix\servicionotificaciones\src\test\resources
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ servicionotificaciones ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 1 source file with javac [debug parameters release 21] to target\test-classes
[INFO] --- surefire:3.5.5:test (default-test) @ servicionotificaciones ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.smartlogix.servicionotificaciones.NotificacionServiceImplTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build
as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: Java agent has been loaded dynamically (C:\Users\anaia\m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disabled by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.029 s -- in com.smartlogix.servicionotificaciones.NotificacionServiceImplTest
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 9.896 s
[INFO] Finished at: 2026-06-16T17:53:26-04:00
[INFO]
[INFO] -----
```

9. Evidencia de versionamiento y branching en GitHub

El equipo utilizó repositorios colaborativos en GitHub y una estrategia de ramas feature para separar los avances por componente. Esta evidencia respalda el trabajo colaborativo, el control de cambios, los commits descriptivos y la publicación de ramas remotas para revisión mediante Pull Request.

La metodología aplicada consistió en clonar el repositorio correspondiente, crear una rama feature, agregar los cambios con git add, confirmar mediante git commit y subir la rama al repositorio remoto mediante git push origin.

```

MINGW64 ~/Frontend-Smartlogix-Aravena-Manriquez-Corral
$ git switch -c feature/login-registro-app
Switched to a new branch 'feature/login-registro-app'

MINGW64 ~/Frontend-Smartlogix-Aravena-Manriquez-Corral (feature/login-registro-app)
$ git add .
warning: in the working copy of 'App.jsx', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'Login/Login.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'Login/Login.jsx', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'Registro/Registro.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'Registro/Registro.jsx', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it

MINGW64 ~/Frontend-Smartlogix-Aravena-Manriquez-Corral (feature/login-registro-app)
$ git commit -m "Feat: agrego login registro y configuracion principal app"
[feature/login-registro-app e519a82] feat: agrego login registro y configuracion principal app
6 files changed, 719 insertions(+)
create mode 100644 App.jsx
create mode 100644 Login/Login.css
create mode 100644 Login/Login.jsx
create mode 100644 Registro/Registro.css
create mode 100644 Registro/Registro.jsx
create mode 100644 index.html

MINGW64 ~/Frontend-Smartlogix-Aravena-Manriquez-Corral (feature/login-registro-app)
$ git push origin feature/login-registro-app
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 4.26 KiB | 2.13 MiB/s, done.
Total 10 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/login-registro-app' on GitHub by visiting:
remote:   https://github.com/anaqueso/Frontend-Smartlogix-Aravena-Manriquez-Corral/pull/new/feature/login-registro-app
remote:
To https://github.com/anaqueso/Frontend-Smartlogix-Aravena-Manriquez-Corral.git
 * [new branch]      feature/login-registro-app -> feature/login-registro-app

MINGW64 ~/Frontend-Smartlogix-Aravena-Manriquez-Corral (feature/login-registro-app)
$
  
```

Figura 9.1. Evidencia de branching y push del frontend en la rama feature/login-registro-app.

```
kamik@Ninita MINGW64 ~
$ cd ~/Downloads

kamik@Ninita MINGW64 ~/Downloads
$ git clone https://github.com/anaqueso/Documentacion-Smartlogix
Cloning into 'Documentacion-Smartlogix'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 1), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), 1.56 MiB | 4.41 MiB/s, done.
Resolving deltas: 100% (1/1), done.

kamik@Ninita MINGW64 ~/Downloads
$ cd Documentacion-Smartlogix

kamik@Ninita MINGW64 ~/Downloads/Documentacion-Smartlogix (main)
$ git switch -c feature/README-General
Switched to a new branch 'feature/README-General'

kamik@Ninita MINGW64 ~/Downloads/Documentacion-Smartlogix (feature/README-General)
$ git add .
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

kamik@Ninita MINGW64 ~/Downloads/Documentacion-Smartlogix (feature/README-General)
$ git commit -m "Feat: agrego README general"
[feature/README-General 7872dd7] feat: agrego README general
1 file changed, 605 insertions(+), 1 deletion(-)

kamik@Ninita MINGW64 ~/Downloads/Documentacion-Smartlogix (feature/README-General)
$ git push origin feature/README-General
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 5.34 KiB | 5.34 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/README-General' on GitHub by visiting:
remote:   https://github.com/anaqueso/Documentacion-Smartlogix/pull/new/feature/README-General
remote:
To https://github.com/anaqueso/Documentacion-Smartlogix
 * [new branch]   feature/README-General -> feature/README-General

kamik@Ninita MINGW64 ~/Downloads/Documentacion-Smartlogix (feature/README-General)
$ |
```

Figura 9.2. Evidencia de branching y push del repositorio de documentación en la rama feature/README-General.

```
danae@LAPTOP-NSV0ER1Q MINGW64 ~
$ cd ~/Downloads

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads
$ git clone https://github.com/anaqueso/Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral.git
Cloning into 'Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral'...
remote: Enumerating objects: 280, done.
remote: Counting objects: 100% (280/280), done.
remote: Compressing objects: 100% (182/182), done.
remote: Total 280 (delta 80), reused 231 (delta 36), pack-reused 0 (from 0)
Receiving objects: 100% (280/280), 72.07 KiB | 868.00 KiB/s, done.
Resolving deltas: 100% (80/80), done.

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads
$ cd Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral (main)
$ git switch -c feature/docker-compose
Switched to a new branch 'feature/docker-compose'

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral (feature/docker-compose)
$ git add .

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral (feature/docker-compose)
$ git commit -m "Feat: agrego docker-compose con infraestructura completa"
[feature/docker-compose 84a74cf] feat: agrego docker-compose con infraestructura completa
1 file changed, 283 insertions(+),
create mode 100644 docker-compose.yml

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral (feature/docker-compose)
$ git push origin feature/docker-compose
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.40 KiB | 477.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/docker-compose' on GitHub by visiting:
remote:   https://github.com/anaqueso/Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral/pull/new/feature/docker-compose
remote:
To https://github.com/anaqueso/Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral.git
 * [new branch]   feature/docker-compose -> feature/docker-compose

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/Smartlogix_Fullstack_Backend_Aravena_Manriquez_Corral (feature/docker-compose)
$ |
```

Figura 9.3. Evidencia de branching y push del backend en la rama feature/docker-compose.

Estas evidencias demuestran que los componentes frontend, backend y documentación fueron trabajados de forma separada mediante ramas, permitiendo trazabilidad de cambios y revisión antes de integrar al repositorio principal.

10. Evidencia de creación y publicación con Docker

La dockerización del backend permite ejecutar los microservicios de SmartLogix de forma portable y consistente. Las capturas integradas en esta sección evidencian la preparación del proyecto, la autenticación en Docker Hub, la construcción de imágenes por microservicio, la verificación local de imágenes y la publicación de los tags en el repositorio remoto.

Las imágenes fueron construidas usando Dockerfile por componente y etiquetadas bajo el repositorio catrinacorrall/smartlogix-proyecto, manteniendo tags independientes para API Gateway, Eureka Server y los microservicios principales.

Comandos utilizados:

Comando	Propósito
<code>docker login</code>	Autenticación en Docker Hub.
<code>docker build -t catrinacorrall/smartlogix-proyecto:<tag> ./<servicio></code>	Construcción de imagen por microservicio.
<code>docker images</code>	Verificación local de imágenes creadas.
<code>docker push catrinacorrall/smartlogix-proyecto:<tag></code>	Publicación de imágenes en Docker Hub.

```

PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main> cd proyectoSmartLogix
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> dir

Directorio: C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix

Mode                LastWriteTime         Length Name
----                -
d-----          10/06/2026   16:59             .github
d-----          01/06/2026   20:12             .vscode
d-----          07/06/2026   17:44            apigateway
d-----          10/06/2026   21:43            eureka-server
d-----          10/06/2026   22:33            servicioenvios
d-----          10/06/2026   22:32            servicioinventario
d-----          10/06/2026   22:34            servicio-notificaciones
d-----          10/06/2026   22:32            servicio-pedidos
d-----          10/06/2026   22:33            servicio-usuarios
-a-----          12/06/2026   18:43           7228 docker-compose.yml
-a-----          10/06/2026   21:27           805 pom.xml
-a-----          02/06/2026   20:09          9071 README.md

PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix>

```

Figura 10.1. Estructura del proyecto backend y carpetas de microservicios utilizadas para construir las imágenes Docker.

```
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker login
Authenticating with existing credentials... [Username: catrinacorral]

! Info → To Login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> █
```

Figura 10.2. Autenticación correcta en Docker Hub mediante docker login.


```
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker build -t catrinacorral/smartlogix-proyecto:eurekaserver-v1 .\eurekaserver
[+] Building 7.6s (16/16) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.2s
=> => transferring dockerfile: 312B                               0.1s
=> [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-21 1.9s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 1.9s
=> [auth] library/maven:pull token for registry-1.docker.io      0.0s
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 2B                                     0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f471191446ddd9cda5d60d 0.2s
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f471191446ddd9cda5d60dab40f0e1 0.1s
=> [internal] load build context                                  0.1s
=> => transferring context: 750B                                    0.1s
=> [build 1/5] FROM docker.io/library/maven:3.9-eclipse-temurin-21@sha256:d7e7f57407437c014571f1ad5a9955f03fc3edcb 0.2s
=> => resolve docker.io/library/maven:3.9-eclipse-temurin-21@sha256:d7e7f57407437c014571f1ad5a9955f03fc3edcb1d9640 0.2s
=> CACHED [stage-1 2/3] WORKDIR /app                             0.0s
=> CACHED [build 2/5] WORKDIR /app                                0.0s
=> CACHED [build 3/5] COPY pom.xml .                               0.0s
=> CACHED [build 4/5] COPY src ./src                              0.0s
=> CACHED [build 5/5] RUN mvn clean package -DskipTests           0.0s
=> CACHED [stage-1 3/3] COPY --from=build /app/target/*.jar app.jar 0.0s
=> exporting to image                                             3.2s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:c820982777c169bacefab32b371941172bf07897d5d3ca54d6596a8929429ec0 0.1s
=> => exporting config sha256:ff5c23b5e00b405901afe720231b6d75a0e8face4aa4539274296475cb213f37 0.1s
=> => exporting attestation manifest sha256:d9a69f9c8c8682bd10bcd5a5241a199b4fb50b7c1687f3a1c3f25afded2b5a62 0.2s
=> => exporting manifest list sha256:c5b7b864caddf844f9aa7b0dff9acf3fbb1d0f4af524684073a0fe5cf2d39dec 0.1s
=> => naming to docker.io/catrinacorral/smartlogix-proyecto:eurekaserver-v1 0.1s
=> => unpacking to docker.io/catrinacorral/smartlogix-proyecto:eurekaserver-v1 2.3s
```

Figura 10.3. Construcción de la imagen Docker del Eureka Server con tag eurekaserver-v1.

```
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker build -t catrinacorral/smartlogix-proyecto:apigateway-v1 .\apigateway
[+] Building 44.9s (10/13)                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.2s
=> => transferring dockerfile: 312B                               0.1s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 1.4s
=> [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-21 1.4s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 2B                                     0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f471191446ddd9cda5d60d 0.1s
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f471191446ddd9cda5d60dab40f0e1 0.1s
=> [build 1/5] FROM docker.io/library/maven:3.9-eclipse-temurin-21@sha256:d7e7f57407437c014571f1ad5a9955f03fc3edcb 0.2s
=> => resolve docker.io/library/maven:3.9-eclipse-temurin-21@sha256:d7e7f57407437c014571f1ad5a9955f03fc3edcb1d9640 0.1s
=> [internal] load build context                                  0.6s
=> => transferring context: 15.03kB                                0.4s
=> CACHED [build 2/5] WORKDIR /app                                0.0s
=> [build 3/5] COPY pom.xml .                                     2.5s
=> [build 4/5] COPY src ./src                                     0.6s
=> [build 5/5] RUN mvn clean package -DskipTests                 38.9s
```

Figura 10.4. Construcción de la imagen Docker del API Gateway con tag apigateway-v1.

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/xoxn1sjem7jupnbjpm62qpwjg
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker build -t catrinaco
rral/smartlogix-proyecto:usuarios-v1 ./serviciousuarios
[+] Building 7.8s (12/15)                                docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.1s
=> => transferring dockerfile: 312B                      0.0s
=> [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-21 1.9s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 1.9s
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io 0.0s
=> [auth] library/maven:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                        0.1s
=> => transferring context: 2B                          0.0s
=> [build 1/5] FROM docker.io/library/maven:3.9-eclipse-temurin-21@sha256:d7e7f57407437c014571f1ad5a9955f03fc3edcb 0.2s
=> => resolve docker.io/library/maven:3.9-eclipse-temurin-21@sha256:d7e7f57407437c014571f1ad5a9955f03fc3edcb1d9640 0.1s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f471191446ddd9cda5d60d 0.2s
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f471191446ddd9cda5d60dab40f0e1 0.1s
=> [internal] load build context                        0.1s
=> => transferring context: 21.47kB                      0.1s
=> CACHED [build 2/5] WORKDIR /app                    0.0s
=> [build 3/5] COPY pom.xml .                          1.2s
=> [build 4/5] COPY src ./src                        0.2s
=> [build 5/5] RUN mvn clean package -DskipTests      3.8s
```

Figura 10.5. Construcción de la imagen Docker del microservicio de Usuarios.

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/srwby19dr82nl10yqgfep5o
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker build -t catrinacorral/smartlogix-proyecto:envios-v1 ./servicioenvios
[+] Building 13.7s (12/15)                                docker:desktop-linux
=> [internal] load build definition from Dockerfile        0.2s
=> => transferring dockerfile: 312B                        0.1s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 1.4s
=> [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-21 1.4s
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io 0.0s
=> [auth] library/maven:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                          0.1s
=> => transferring context: 2B                             0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f471191446ddd9cda5d60d 0.1s
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f471191446ddd9cda5d60dab40f0e1 0.1s
=> [internal] load build context                          0.1s
=> => transferring context: 27.49kB                       0.1s
=> [build 1/5] FROM docker.io/library/maven:3.9-eclipse-temurin-21@sha256:d7e7f57407437c014571f1ad5a9955f03fc3edcb 0.1s
=> => resolve docker.io/library/maven:3.9-eclipse-temurin-21@sha256:d7e7f57407437c014571f1ad5a9955f03fc3edcb1d9640 0.1s
=> CACHED [build 2/5] WORKDIR /app                       0.0s
=> [build 3/5] COPY pom.xml .                             0.3s
=> [build 4/5] COPY src ./src                            0.2s
=> [build 5/5] RUN mvn clean package -DskipTests         9.6s
```

Figura 10.6. Construcción de la imagen Docker del microservicio de Envíos.

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/wssnxiroa55rekveqnofsw112
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
catrinacorral/smartlogix-proyecto:apigateway-v1	651ee877aa37	385MB	121MB	
catrinacorral/smartlogix-proyecto:envios-v1	a96ea739f7d8	460MB	156MB	
catrinacorral/smartlogix-proyecto:eurekaserver-v1	c5b7b864cadd	399MB	127MB	
catrinacorral/smartlogix-proyecto:inventario-v1	dc29fb7dc566	461MB	157MB	
catrinacorral/smartlogix-proyecto:notificaciones-v1	1332c513231c	463MB	158MB	
catrinacorral/smartlogix-proyecto:pedidos-v1	c7e6c35a3b4b	469MB	160MB	
catrinacorral/smartlogix-proyecto:usuarios-v1	6be3938c2d3e	452MB	152MB	

```
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix>
```

Figura 10.7. Verificación local de imágenes creadas mediante docker images, mostrando los tags de los microservicios principales.

```
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker push catrinacorral/smartlogix-proyecto:eurekaserver-v1
The push refers to repository [docker.io/catrinacorral/smartlogix-proyecto]
c5a4b8fbd4f8: Pushed
6e338134b21e: Pushed
cae3ac6b796a: Pushed
6a0ac1617861: Mounted from library/postgres
5ef4bf1bda35: Pushed
a3e33baf40c5: Pushed
42516208460b: Pushed
2d1a277377d1: Pushed
eurekaserver-v1: digest: sha256:c5b7b864caddf844f9aa7b0dff9acf3fbb1d0f4af524684073a0fe5cf2d39dec size: 856
```

Figura 10.8. Publicación de la imagen eurekaserver-v1 en Docker Hub mediante docker push.

```
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker push catrinacorrall/smartlogix-proyecto:apigateway-v1
The push refers to repository [docker.io/catrinacorrall/smartlogix-proyecto]
6a0ac1617861: Layer already exists
5ef4bf1bda35: Layer already exists
6e338134b21e: Layer already exists
56cf6f5cde9c: Pushed
2d1a277377d1: Layer already exists
c5a4b8fbd4f8: Layer already exists
42516208460b: Layer already exists
7e47733a69db: Pushed
apigateway-v1: digest: sha256:651ee877aa37abd23e13a21a3c6076be5680b0efeb7ccbe5c6b21a472f9f52dc size: 856
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix>
```

Figura 10.9. Publicación de la imagen apigateway-v1 en Docker Hub mediante docker push.

```
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix> docker push catrinacorrall/smartlogix-proyecto:pedidos-v1
The push refers to repository [docker.io/catrinacorrall/smartlogix-proyecto]
c5a4b8fbd4f8: Layer already exists
b1373e50ee19: Pushed
6a0ac1617861: Layer already exists
6e338134b21e: Layer already exists
42516208460b: Layer already exists
5ef4bf1bda35: Layer already exists
2d1a277377d1: Layer already exists
cd27a3e86b4c: Pushed
pedidos-v1: digest: sha256:c7e6c35a3b4b484d0b23319537cb508bcfec9ab2cbac933a87a6bc67c765ae1 size: 856
PS C:\Users\danae\Documents\smartlogix_backend_aravena_corral_manriquez-main\proyectoSmartLogix>
```

Figura 10.10. Publicación de la imagen pedidos-v1 en Docker Hub mediante docker push.

Tag	OS	Type	Pulled	Pushed
notificaciones-v1	linux/amd64	Image	less than 1 day	1 minute
envios-v1	linux/amd64	Image	less than 1 day	2 minutes
pedidos-v1	linux/amd64	Image	less than 1 day	3 minutes
inventario-v1	linux/amd64	Image	less than 1 day	4 minutes
usuarios-v1	linux/amd64	Image	less than 1 day	5 minutes

Figura 10.11. Evidencia visual en Docker Hub con las imágenes/tags publicados del proyecto SmartLogix.

Con estas evidencias se confirma que el backend fue dockerizado y que las imágenes fueron generadas y publicadas correctamente. Esto respalda la arquitectura de contenedores definida para SmartLogix y permite replicar el entorno en otros equipos mediante imágenes versionadas.

Bibliografía

Mecalux. (s/f). La arquitectura de microservicios y sus ventajas en el desarrollo de software.

Mecalux.es. Recuperado el 3 de abril de 2026, de <https://www.mecalux.es/blog/microservicios-arquitectura>

Miralpeix, D. (2026, enero 7). Qué es una arquitectura de microservicios. Flexygo; CEESi Asesores. <https://www.flexygo.com/que-es-una-arquitectura-de-microservicios/>

Gil, S. (2020, diciembre 11). Cuándo elegir microservicios para construir tu e-commerce. Blog de hiberus; Hiberus. <https://www.hiberus.com/crecemos-contigo/cuando-elegir-microservicios-para-construir-tu-e-commerce/>

de Yavé, V. M. C. (s/f). Patrones de arquitectura en microservicios: configuración....

Paradigmadigital.com. Recuperado el 3 de abril de 2026, de

<https://www.paradigmadigital.com/dev/patrones-arquitectura-microservicios-configuracion-externalizada/>

Susnjara, S., & Smalley, I. (2025, noviembre 27). Patrones de diseño para microservicios. Ibm.com.

<https://www.ibm.com/mx-es/think/topics/microservices-design-patterns>

P of EAA. (s/f). Martinowler.com. Recuperado el 3 de abril de 2026, de

<https://martinowler.com/books/ea.html>

Building Microservices, 2nd Edition. (s/f). Samnewman.io. Recuperado el 3 de abril de 2026, de

https://samnewman.io/books/building_microservices_2nd_edition/

Amazon Web Services. (s.f.). Database-per-service patrón.

https://docs.aws.amazon.com/es_es/prescriptive-guidance/latest/modernization-data-persistence/database-per-service.html

Mostapha, M. (2025, 10 de abril). Microservicios con Spring Cloud - Arquitectura completa.

<https://www.mostapha.dev/blog/microservicios-spring-cloud-arquitectura-completa>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Nygard, M. T. (2018). *Release It!: Design and deploy production-ready software*. Pragmatic Bookshelf.

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns*. Addison-Wesley.

Conclusión

El trabajo realizado en esta tercera etapa permitió implementar y validar la arquitectura de microservicios de SmartLogix en su totalidad. La selección de patrones como Repository Pattern, Factory Method, Circuit Breaker y Database per Service respondió directamente a los requerimientos del caso, permitiendo construir una solución modular, escalable y desacoplada.

La incorporación de contenedores Docker mediante Docker Compose establece una base sólida para el despliegue del sistema, garantizando consistencia entre entornos. Además, las consideraciones de seguridad, privacidad y sostenibilidad integradas desde el diseño inicial aseguran una solución eficiente, confiable y alineada con las necesidades del cliente.

En esta etapa, la arquitectura fue implementada en su totalidad mediante Spring Boot, Docker Compose, PostgreSQL y React, validando las decisiones arquitectónicas mediante pruebas unitarias con cobertura mínima del 60%.